Pokédex Application

Midterm Exam | CS Elective 2: Mobile Development with Flutter

---

Instructions:

You will extend your existing Pokédex project, which already has async fetching, setState, and state management using the Provider package. For this examination, you will implement five features and manage the Pokémon list and related functionality using ONE new state management approach of your choice: Riverpod or BLoC.

Your application must implement the following:

1. **Pokémon List Management**
    - Move the Pokémon list loading and fetching logic into your chosen state management class.
    - Fetch Pokémon data from the API asynchronously.
    - Display appropriate UI states while data is loading, successfully loaded, or has encountered an error.
    - Allow users to refresh the Pokémon list from the home page. (this should not create new method, just RECALL the existing fetch() method via the state management)

2. **Pokémon Nickname Editor**
    - Allow users to assign a custom nickname to a Pokémon.
    - Allow users to edit or remove an existing nickname.
    - Store nicknames in app state using your chosen state management approach.
    - Display the custom nickname on the corresponding Pokémon card or detail screen, while retaining the Pokémon's original name.

3. **Favorite Pokémon/s**
    - Allow users to add or remove Pokémon from their favorites.
    - Display favorite Pokémon in a retractable tab or panel (possibly in AppBar) that users can expand and collapse all throughout the app.
    - Update the favorites panel immediately when a Pokémon is added or removed.

4. **Favorite Indicator**
    - Display a small heart icon on each Pokémon card to indicate whether it is a favorite.
    - The heart icon must reflect the Pokémon's current favorite status.

- Tapping the heart must add or remove the Pokémon from favorites and update the UI accordingly.

5. **Pokémon Type Filter**
   - Add a filtering mechanism based on Pokémon type, such as Fire, Water, Grass, Electric, or other available types.
   - Allow users to select a type and display only Pokémon matching that type.
   - Provide an option to clear the filter and display the full Pokémon list again.

Choose your State Management:

Choose **ONLY 1** state management package from these two 3rd-party packages, and build the app from here onward.

1. ## Riverpod
   a. https://riverpod.dev/docs/introduction/getting_started
   b. https://pub.dev/packages/riverpod

2. ## BLoC
   a. https://pub.dev/packages/flutter_bloc/changelog
   b. https://bloclibrary.dev/getting-started/

UI Requirements

Your application must provide the following UI elements:

1. **Home Page**
   - Displays the Pokémon list.
   - Shows a loading indicator while fetching data.
   - Shows a meaningful error message if fetching fails.
   - Supports pull-to-refresh or an equivalent refresh action.
   - Provides access to the type filter and favorites panel.
2. **Pokémon Cards**
   - Display the Pokémon's image and original name.
   - Display the custom nickname when one has been assigned.
   - Include a heart icon indicating favorite status.
   - Update the card immediately when the nickname or favorite status changes.
3. **Nickname Editor**
   - Provides a text input for entering or editing a nickname.
   - Provides an action to save the nickname.
   - Provides an option to remove the nickname.
4. **Favorites Panel**
   - Can be expanded and collapsed.
   - Displays only the Pokémon marked as favorites.

    - Updates immediately when favorites are added or removed.
    - Displays an appropriate message when no Pokémon have been favorited.
5. **Type Filter**
    - Provides a dropdown, chip selector, or another suitable selection control.
    - Filters Pokémon by their actual type data.
    - Includes an option to clear the filter.

General Guidelines:

- Handle the functionalities as methods on your chosen state management.
- Use the Pokémon API and data source already used in your existing project unless otherwise instructed.
- Follow Flutter conventions for widget composition, asynchronous operations, and state management.
- Handle API errors and empty results gracefully.
- Dispose of controllers and other resources where appropriate.
- Your output will be graded based on how well you understand the logic of your app with state management and how well you were able to apply it.

Submission:

1. Deadline will be next Thursday, `Oct 22, 2026`, 11:59PM.
2. There will be 2 submissions:
    a. GitHub repo
        i. Branch out from your latest Pokedex activity branch and name it **midterm-exam**.
        ii. Push your code.
    b. Video Explanation
        i. Should not be more than 15 minutes.
        ii. Explain the ff:
            1. Your code and how you did the application.
            2. Significance of using state management in general and in the context of your app.
            3. Why you chose your chosen state management approach.